Student guide: exact calculations and the custody of evidence

Learning companion for Paper 004, v0.1.0. Read this before the quiz. Solutions are in the separate instructor key. You need basic vectors, sums, absolute values, and the idea of a cryptographic digest. No Rust or Lean installation is needed for the hand work.

By the end, you should be able to calculate the paper's finite fixture, distinguish a mathematical witness from a hash commitment, predict several typed refusals, and state what evidence a reproducible native run would require.

1. Five questions that belong to different checks

Question	Relevant evidence	What remains separate
Are these the committed bytes?	Named encoding profile and comparison with a retained digest anchor	Whether the bytes state a true claim
Is this the declared finite problem?	Vertices, oriented edges, faces, coefficient domain, weights and manifest anchor	Whether it models an external system faithfully
Does this candidate equal the replayed normal form?	Exact solve, replay and candidate comparison	Whether the producer is authentic
Did the intended executable produce this result?	Independently trusted binary selection and execution/provenance evidence	Whether source-to-binary construction is reproducible
May this result be admitted?	The action's conformance and attestation policy	Merely obtaining zero numerical defects

Avoid carrying an answer from one row into another. A hash can bind a false statement. A correct calculation can have refused admission. An unsigned outcome can be internally consistent without establishing its producer.

2. Reconstruct the carrier before calculating

Use vertices `0,1,2,3`, oriented edges in the order

`(0,1), (1,2), (2,0), (1,3), (3,0)`

and faces `(0,1,2)` and `(0,1,3)`. Unit edge penalties are used. For a vertex potential `f`, the edge difference on `(u,v)` is `f(v)-f(u)`.

The before cochain is `c=(3,2,2,1,1)`. In this fixture:

$$\Phi_1(c) = \min_f \sum_e |c_e - (d_0 f)_e|, \qquad \Gamma_2(c) = |c_{01} + c_{12} + c_{20}| + |c_{01} + c_{13} + c_{30}|.$$

The faces are data, not decorations on the graph. A graph with the same edges but one fewer face changes `d_1` and therefore changes what the face check observes. Its carrier identity must change.

3. A worked upper bound and a matching lower bound

Try `f=(0,-2,-2,-1)`. The calculation is:

Edge	Before `c`	Difference `d f`	Residual `c-d f`
01	3	-2	5
12	2	0	2
20	2	2	0
13	1	1	0
30	1	1	0

The residual cost is `7`. This establishes `Phi1 7`; trying one potential does not yet prove optimality.

For the lower bound, assign `y=(1,1,1,0,0)` to the oriented edges. It circulates around `0→1→2→0`: inflow equals outflow at each vertex. Thus `y·d g=0` for every potential `g`. Each coordinate has absolute value at most its unit penalty, so

$$\sum_e |c_e - (d_0 g)_e| \geq y \cdot (c - d_0 g) = y \cdot c = 7.$$

Upper and lower bounds meet. **Phi1=7 exactly.** No timing result, approximate optimizer, or native receipt is needed to verify this small mathematical fact.

The two face sums are 7 and 5, hence Gamma2=12. The graph has `5-4+1=2` independent cycles, and these two faces have independent boundaries spanning them. Under the paper's declared solve, the final normal form is zero; its final Phi1 and Gamma2 are both zero. This does not make the original vector `c` a gradient. It distinguishes the before calculation from the normal form returned by the solve.

4. Exact execution can end in refused admission

For the canonical host fixture, the expected execution label is `EXECUTED_REPLAYED_EXACT_LAB_EVIDENCE`. The candidate matches the replayed normal form, and the process consumes two internal permits, for the letters `E,T`.

The admission result is still `Refused`: conformance is `Partial`, three laws are unchecked, and attestation is `unsigned_lab`. The codes are `PRIME000_CONFORMANCE_PARTIAL` and `UNSIGNED_LAB_ATTESTATION`.

A good display keeps execution, numerical summaries, conformance, attestation and admission together. Printing "verified: zero" loses information the consumer needs.

5. Canonical bytes need a named contract

The paper names three layers:

Layer	Rules to remember
Domain canonical algebra	Tagged types: integer `7` and string `"7"` remain distinct
`PacketCodec` helper	NFC normalization, then tagged domain encoding; its `verify` checks payload digest and authority, not every sealed envelope field
Restricted host's outer JSON	`omega-sorted-json-no-float-v1`; exact arithmetic coefficients use minimal i128 strings

RFC 8785 has its own primitive serialization and sorting rules and preserves Unicode string data as supplied. Consider U+00E9 (`é`) and U+0065 U+0301 (`e` followed by a combining acute accent). They look alike but their direct UTF-8 bytes differ. NFC maps both to U+00E9. A normalization policy is therefore part of the meaning of a digest comparison.

The type of a mathematical quantity and its permitted wire spelling are separate. The restricted host computes internally with `BigRational`, but its before/candidate wire fields accept integer strings. Internal rational arithmetic does not make `"1/2"` a permitted spelling in those fields. The broader dispatcher has distinct action contracts.

6. Three ways a plausible receipt can mislead

A changed carrier with a stale anchor. Recomputing an outer request hash does not repair a mismatch between a derived manifest and its independently bound expected carrier hash.

A changed candidate with matching surrounding hashes. The host still compares its values to the replayed normal form. A candidate digest cannot replace that comparison.

A changed outcome with a recomputed root. The unkeyed root can become consistent with the edited bytes. That alone supplies neither a fresh native replay nor producer authentication. If an attacker can also replace the expected anchor, a comparison against that replacement provides no independent protection.

Write down which party controls each value and which anchor the checker actually trusts. "It has a hash" is not a threat model.

7. Interface and resource boundaries

The default host, runtime dispatcher and foundation library are distinct. The environment override `OMEGA_SYMBOL_GAME_HOST` belongs to the host seam. The graph tutorial adapter instead needs `OMEGA_FUNCTIONAL_GRAPH_BIN` and `OMEGA_FUNCTIONAL_GRAPH_SHA256`. Advertising an action does not configure another executable.

The restricted host has a 1,048,576-byte request ceiling, dimension limits of 4,096, and four dense-matrix cell checks at 1,048,576 cells each. The dispatcher uses a separate 65,536-byte input ceiling. Its bounded reader takes at most its limit plus one witness byte.

The declared timeout is validated metadata, not an in-process watchdog. A process wrapper must separately enforce and record any required wall-clock termination. Finite dimensions do not bound the bit complexity of all exact intermediate arithmetic.

In the package governor, an unset graph BIN chooses exact Python enumeration. A configured native call that refuses or fails must propagate that outcome. A native-required engine action cannot be relabeled as successful native execution by replacing it with an offline Python answer.

8. Read test receipts as bounded evidence

Retain the command, date, exit status, source/input hashes, interpreter/toolchain, native selection and binary digest when used, and passed/failed/skipped counts. A scripted executable checks adapter behavior; it does not independently validate native arithmetic. Two wrappers calling one solver are not two independent solvers.

The package's Lean modules concern specific finite mathematical statements. They do not prove the host protocol or connect every transport field to the implementation. An axiom census records named declarations and their axiom use, not the truth of every sentence in a paper.

Now attempt the quiz. Use the hints after an honest first attempt. For each answer, name the carrier, calculation/check, and limit of the resulting claim.